BRIEFING TÉCNICO · DOCUMENTAÇÃO DE SISTEMA

Monitor de Pragas

Plataforma de monitoramento de armadilhas e sensores IoT (Tuya) para controle de pragas — como o sistema é construído e como ele funciona, por dentro.

Produto: Monitor de Pragas (PWA) · **Contexto:** Control Up · **URL:** sistemas.controlup.com.br/monitordepragas
Natureza: Documento técnico de arquitetura e funcionalidades · **Versão do documento:** 1.0

Sumário

1. 1. Visão geral
2. 2. Arquitetura (três camadas)
3. 3. Integração com a Tuya (núcleo do sistema)
4. 4. Notificações: primeiro plano e push em segundo plano
5. 5. Câmera e vídeo ao vivo
6. 6. PWA (aplicativo instalável)
7. 7. Modelo de dados: empresas e dispositivos
8. 8. Funcionalidades do aplicativo (telas)
9. 9. Módulo NFC: confirmação de troca em campo
10. 10. Mapa de arquivos e endpoints
11. 11. Dependências e requisitos operacionais
12. 12. Limitações técnicas conhecidas

1. Visão geral

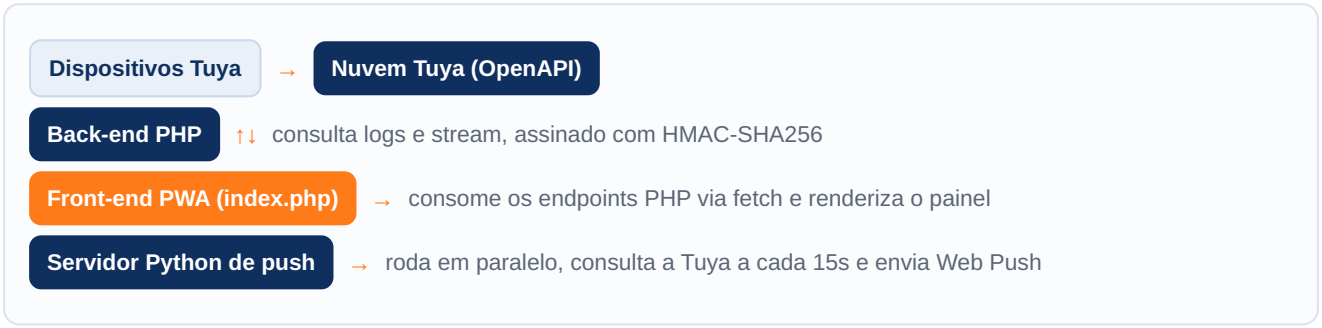
O **Monitor de Pragas** é um aplicativo web instalável (PWA) que centraliza o monitoramento de armadilhas e sensores de controle de pragas. Os dispositivos físicos são equipamentos IoT da plataforma **Tuya** (sensores de presença/PIR e câmeras de armadilha luminosa). O sistema lê os eventos desses dispositivos na nuvem da Tuya, mostra o status num painel, dispara notificações de movimento (inclusive com o app fechado) e organiza as empresas atendidas, seus dispositivos e as trocas de manutenção feitas em campo.

Tecnicamente, é um sistema **sem framework e sem banco de dados**: a interface é renderizada em JavaScript puro no próprio `index.php`, o back-end são scripts PHP que funcionam como *proxy* assinado para a API da Tuya, e o estado é guardado em `localStorage` (no navegador) e em arquivos `.json` (no servidor). As notificações reais em segundo plano são entregues por um serviço **Python** que roda continuamente.

> **Stack em uma linha**
>
> **Front:** PWA (HTML/CSS/JS puro, Service Worker, Web Push) | **Back:** PHP 8 (proxy Tuya via cURL + HMAC-SHA256) | **Push:** Python (pywebpush + VAPID, polling 15s) | **IoT:** Tuya OpenAPI (data center US) | **Vídeo:** HLS | **Persistência:** localStorage + arquivos JSON

2. Arquitetura (três camadas)

O sistema se divide em três camadas que se comunicam por HTTP e por arquivos no servidor:



Camada 1 — Front-end PWA

Todo o painel vive no `index.php` (~3.975 linhas). É uma aplicação de página única renderizada em JavaScript: as telas são desenhadas por funções de renderização (`renderizarTela` , `renderizarMapaEmpresa` , etc.) e a navegação troca o conteúdo sem recarregar a página. O estado do usuário (empresas cadastradas, layout do mapa, histórico, trocas registradas manualmente) fica no `localStorage` do navegador.

Camada 2 — Back-end PHP (proxy assinado da Tuya)

Os arquivos PHP não guardam regra de negócio pesada: eles autenticam na Tuya, consultam a API e devolvem **só JSON** para o front. Cada endpoint gera o token, assina a requisição e trata a resposta. São eles o `api_logs.php` (eventos de movimento), `api_stream_url.php` (vídeo), `salvar_inscricao_push.php` (inscrição de notificação) e o módulo `nfc/` .

Camada 3 — Servidor Python de push

O `servidor_push.py` é um processo que precisa ficar rodando o tempo todo. Ele repete, a cada 15 segundos, o mesmo tipo de consulta que o PHP faz, compara com os eventos já vistos e, quando acha um movimento novo, envia uma notificação **Web Push** real para todos os celulares inscritos — o que faz a notificação aparecer mesmo com o app fechado ou o celular bloqueado.

3. Integração com a Tuya (núcleo do sistema)

É a parte mais sensível. Toda comunicação usa o padrão de assinatura da Tuya OpenAPI, no data center dos EUA (`openapi.tuyaus.com`).

3.1 Autenticação e assinatura

- **Credenciais do projeto:** `client_id` e `secret` (do projeto de nuvem na Tuya), embutidos no código PHP e Python.
- **Token de acesso:** obtido em `GET /v1.0/token?grant_type=1` ; reaproveitado para as consultas seguintes da mesma rodada.
- **Assinatura (HMAC-SHA256):** monta-se `stringToSign = método + "\n" + sha256(body) + "\n\n" + path` . Sem token, assina-se `client_id + t + stringToSign` ; com token, `client_id + access_token + t + stringToSign` . O resultado vai em maiúsculas no cabeçalho `sign` , junto de `client_id` , `t` (timestamp em ms) e `sign_method: HMAC-SHA256` .

3.2 Leitura de eventos (logs do dispositivo)

O endpoint central é `GET /v2.0/cloud/thing/{device_id}/logs` , com janela de tempo (`start_time` / `end_time` em ms) e um parâmetro `type` que define a categoria do log:

Tipo de dispositivo	type	Lógica de filtro
Câmera (armadilha luminosa)	`1`	Eventos de movimento da câmera; tenta extrair URL de imagem do log (`image_url` , `attribute` , etc.).
PIR (porta-isca, sensor Zigbee)	`7`	Mantém apenas `code == "pir"` e `value == "pir"` . Cada detecção gera um par *pir* (começou) + *none* (parou); o *none* é descartado para não notificar duas vezes.

A janela de busca é de **1 hora** para trás, e os eventos são ordenados do mais recente para o mais antigo antes de voltar ao front.

Pré-requisito crítico: IoT Core autorizado

Esse endpoint de logs pertence ao serviço **IoT Core** da Tuya. Se o projeto de nuvem não estiver assinado e **autorizado** para esse serviço (ou se o período de teste expirar), a API responde `28841105 – "No permissions. This project is not authorized to call this API"` e nenhum evento chega ao sistema, mesmo o dispositivo estando online e notificando no app da Tuya. A retenção padrão de logs do IoT Core é de 7 dias, o que é suficiente (a busca é de 1 hora).

4. Notificações: primeiro plano e push em segundo plano

Existem **dois caminhos independentes** de notificação, e entender a diferença é essencial.

4.1 App aberto (primeiro plano) — *pull*

Com o painel aberto, o JavaScript chama `buscarEventos()` , que faz `fetch` no `api_logs.php` periodicamente. Eventos novos (não vistos antes, por `event_time`) viram **toasts** na tela. Isso depende do app estar aberto e não envolve push.

4.2 App fechado (segundo plano) — *push real*

A notificação com o app fechado é entregue por Web Push, e exige a cadeia completa funcionando:

Navegador → **salvar_inscricao_push.php** → **push_subscriptions.json**

O usuário toca em "Ativar notificações"; o navegador cria uma inscrição com a chave pública VAPID e o PHP a salva no servidor.

servidor_push.py (loop 15s) → **Web Push (VAPID)** → **Service Worker (sw.js)** → **Notificação**

O Python acha o evento novo, envia o push assinado; o Service Worker recebe no evento `push` e exibe a notificação.

- **Chaves VAPID:** par de chaves — a **pública** fica no `index.php` (usada pelo navegador ao se inscrever) e a **privada** no `servidor_push.py` (usada para assinar o envio). As duas precisam ser do mesmo par, senão o envio é rejeitado.
- **Controle de duplicidade:** o Python guarda os eventos já notificados em `eventos_ja_notificados.json` (últimos 500) e, na primeira rodada após iniciar, apenas marca os eventos como vistos sem notificar.

- **Inscrições expiradas:** quando o envio retorna 404/410 (*Gone*), a inscrição é removida da lista automaticamente.
- **Dependência:** se o `servidor_push.py` não estiver rodando, não há push em segundo plano — ainda que a Tuya registre tudo.

5. Câmera e vídeo ao vivo

Para a armadilha luminosa (câmera), o sistema oferece vídeo ao vivo. O endpoint `api_stream_url.php` gera o token, chama `POST /v1.0/devices/{device_id}/stream/actions/allocate` pedindo `type: "HLS"` e devolve a URL do stream. No front, `abrirVideoNoEncaixe()` injeta um player HLS no card do dispositivo, com overlay de carregamento e opção de tentar novamente. O endpoint aceita `?device_id=` para suportar várias câmeras no futuro.

6. PWA (aplicativo instalável)

- **manifest.json:** modo `standalone`, orientação retrato, ícones 192/512 (inclusive *maskable*), `start_url` em `index.php`. Permite "instalar na tela inicial".
- **Service Worker (sw.js):** registra-se na instalação, assume o controle das abas (`clients.claim`), deixa as requisições passarem direto para a rede (sem cache offline), e trata os eventos `push` (exibir notificação) e `notificationclick` (focar/abrir o app).
- **iOS:** o Web Push só funciona em iOS 16.4+ e com o app **instalado na tela inicial**; no Safari comum a inscrição é criada mas a notificação não aparece.

7. Modelo de dados: empresas e dispositivos

O catálogo é organizado em **Empresa → Família de pragas → Dispositivo**.

- **Empresas fixas:** definidas direto no JavaScript (`EMPRESAS_FIXAS`). Hoje: **Control Up**, com as famílias *Roedores*, *Aves* e *Insetos*.
- **Dispositivos:** cada item tem `id` (ex.: `porta-isca-1` , `armadilha-1`), `nome` , `tipo` (`sensor` ou `camera`), `ativo` e `deviceId` (o id real na Tuya, quando vinculado).
- **Empresas dinâmicas:** cadastradas pela dedetizadora e salvas no `localStorage` (`empresas_cadastradas`) — ou seja, existem apenas no navegador onde foram criadas.
- **Dimensionamento:** o sistema calcula a quantidade recomendada de dispositivos por metragem (`calcularQuantidadeRecomendada` / `montarItensDaFamilia`).

Dispositivo (id)	Nome	Tipo	Família	deviceId Tuya
porta-isca-1	Porta Isca Mult	sensor (PIR)	Roedores	eb05379e8d20edb316zflg
porta-isca-2	Porta Isca Control Box	sensor	Roedores	— (simulado)
armadilha-1	Armadilha Luminosa Easy Up 15	camera	Insetos	eb8494149add8c35e4skeo

8. Funcionalidades do aplicativo (telas)

Funcionalidade	O que faz (técnico)

Lista de empresas / cadastro	Lista empresas fixas + dinâmicas; formulário para cadastrar nova empresa (salva no localStorage).
Famílias de pragas	Agrupa dispositivos por Roedores/Aves/Insetos, com contagem de dispositivos por família.
Mapa de dispositivos	Posiciona pinos sobre uma planta (coordenadas `mapaX/mapaY`); modo de edição arrasta os pinos; o layout é salvo por empresa no localStorage.
Detalhe do dispositivo	Mostra status, abre vídeo ao vivo (câmera) e o bloco de manutenção; permite ativar/vincular um `deviceId` real.
Controle de manutenção	Trocas de **refil** (alerta em 30 dias) e **lâmpada** (300 dias); card com status "em dia/vencido" e registro manual de data.
Toasts de eventos	Avisos em tela para movimento, consumo (simulado) e manutenção.
Pendências e roteiro do dia	`calcularPendenciasGerais` junta notificações não resolvidas + refil/lâmpada vencidos; monta um roteiro priorizado de visitas.
Tendência / consumo	`calcularTendencia` analisa a frequência de eventos numa janela (ex.: 7 dias).
Históricos	Histórico de eventos por dispositivo e histórico de notificações (ambos no localStorage), com badge e painel lateral.
Catálogo de produtos	Famílias de produtos e fichas técnicas (especificações, intervalos de manutenção).
Relatório de visita	Resumo por empresa (`montarResumoVisita`) e painel de relatório para acompanhamento.

9. Módulo NFC: confirmação de troca em campo

Permite que o técnico, em campo, confirme uma troca (refil, lâmpada, cola Peg Rat, veneno) encostando o celular numa etiqueta NFC. Grava no servidor em arquivo JSON (sem banco), com `LOCK_EX`; o `diagnostico_gravacao.php` testa a permissão de escrita da pasta.

Modelo antigo (baseado em "pontos")

O admin criava "pontos" protegidos por senha (`pontos.php` , `admin_auth.php`), cada etiqueta carregava uma chave secreta, e o técnico confirmava em `checkin.php`. Problema: o `id` do ponto era gerado automaticamente e **não batia** com o id da armadilha no mapa, então a troca não aparecia na página principal. Além disso, a página principal lê as trocas do `localStorage`, enquanto o NFC gravava no **servidor** — dois cofres que não se conversavam.

Modelo novo (recomendado)

- **Catálogo compartilhado no servidor** (`empresas.json`): a fonte que faltava, lida pelas duas pontas.
- **registrar.php:** tela no padrão do sistema — **empresa → dispositivos ativos → troca** — gravando pelo `item_id` real (ex.: `porta-isca-1`), o mesmo do mapa. A etiqueta pode abrir já no dispositivo via `?empresa=…&item=…` .
- **Ponte no index.php:** um trecho busca `nfc/api_servicos.php` e injeta as trocas do servidor nas chaves que o app já lê, fazendo a troca do técnico aparecer na página principal.

10. Mapa de arquivos e endpoints

Arquivo	Papel
`index.php`	Todo o painel PWA (UI, navegação, estado, polling, push no front).
`api_logs.php`	Proxy: eventos de movimento (câmera + PIR) da Tuya, em JSON.
`api_stream_url.php`	Proxy: gera a URL de stream HLS da câmera.
`salvar_inscricao_push.php`	Salva a inscrição de push do navegador em `push_subscriptions.json`.
`servidor_push.py`	Serviço contínuo (15s) que envia Web Push dos eventos novos.
`sw.js` / `manifest.json`	Service Worker (push, clique) e manifesto do PWA.
`diagnostico_pir.php`	Lista os logs `type=7` crus do sensor, sem filtro (diagnóstico).
`nfc/registrar.php`	Tela nova de registro de troca (empresa → dispositivo → troca).
`nfc/registrar_servico.php`	Grava a troca em `servicos.json` pelo item_id real.
`nfc/api_servicos.php`	Histórico e últimas trocas por dispositivo.
`nfc/empresas.json`	Catálogo compartilhado de empresas e dispositivos.
Testes/diagnóstico: `teste_api_pir.php`, `teste_notificacao_pir.php`, `teste_camera.php`, `teste_foto.php`, `teste_stream.php`, `teste_video_vivo.php`, `diagnostico.html`.	

11. Dependências e requisitos operacionais

- **Projeto de nuvem Tuya** com **IoT Core** assinado e autorizado, no data center correto (US). Trial precisa estar válido.
- **Hospedagem PHP 8** com `cURL` e certificados SSL válidos; pasta `nfc/` com permissão de escrita para os arquivos JSON.
- **Processo Python** (`servidor_push.py`) rodando continuamente para o push em segundo plano — idealmente como serviço (systemd: `tuya-push.service`), com `requests` e `pywebpush` instalados.
- **Par de chaves VAPID** consistente entre front (pública) e Python (privada).

12. Limitações técnicas conhecidas

- **Sem banco de dados:** o estado vive em `localStorage` (por navegador/aparelho) e em arquivos JSON no servidor. Dados de um aparelho não aparecem em outro, exceto o que passa pelo servidor.
- **Catálogo duplicado:** as empresas/dispositivos ficam no JS do `index.php` e, no fluxo novo, também no `empresas.json` — os dois precisam ser mantidos em sincronia manualmente.
- **Push frágil por design:** depende do `servidor_push.py` ativo o tempo todo. Uma alternativa mais robusta seria assinar o serviço de mensagens da Tuya (Pulsar) em vez de fazer *polling*.
- **Credenciais no código:** `client_id` / `secret` da Tuya e a chave VAPID privada estão em texto no código PHP/Python.

- **Tipos de manutenção no mapa:** a página principal só exibe card de **refil** e **lâmpada**. Trocas de porta-isca (Peg Rat/veneno) são gravadas, mas ainda não têm card próprio no mapa.
- **iOS:** push exige PWA instalada e iOS 16.4+.

Documento gerado como briefing técnico do sistema Monitor de Pragas (Control Up). Reflete o estado do código analisado, incluindo o módulo NFC reestruturado (empresa → dispositivo → troca).